

Pipes in the Desert
Detailed Plan
E5 - TEAM A

Supervisor:
Dr. Balla Katalin

Members:

Muhammad Ibrahim Shoeb
Arda Gecegörür
İlgin Tunç
Muhammad Hameez Khan
Yahya Akhrikhar
Aasif Mohd

OZLVV3
EKQNAK
TDQWFF
TFBB32
K20UXP
OI6VE6

muhammadibrahimshoeb@gmail.com
ardagecegorur@gmail.com
ilgintunc11@gmail.com
hameezkhan993@gmail.com
yahyaakhrikhar02@gmail.com
aasifuk1122@gmail.com

May 9, 2026

11. User interface specification

11.1 Graphical User Interface

1-Main Menu Screen:

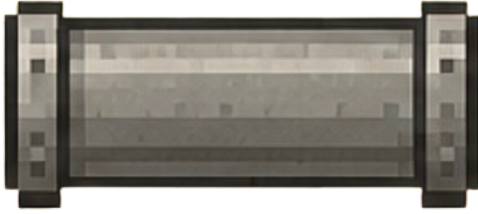


2-Game Setup Screen:

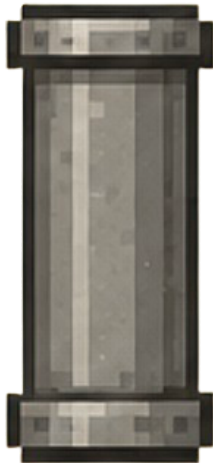


3-Pipes and States of Pipes:

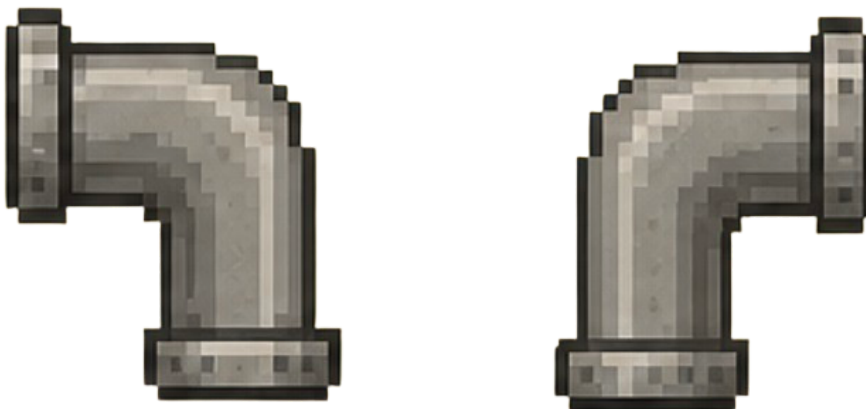
a-Horizontal



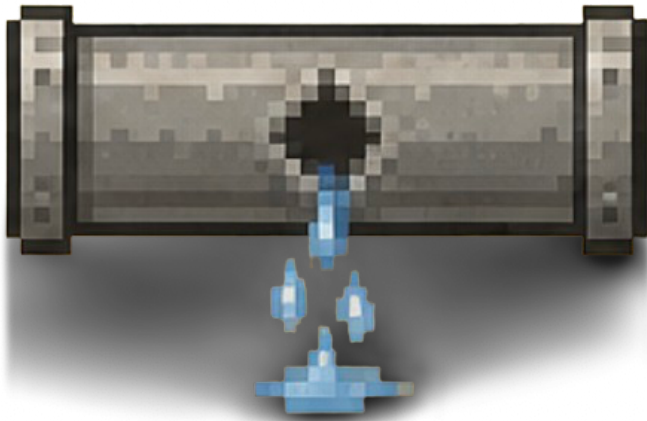
b-Vertical



c-Elbow



d-Punctured Pipe



e-Repaired Pipe



4-Spring:



5-Pump:



6-Cistern



7-Plumber



8-Saboteur



9-Main Gameplay Screen

[What game should approximately look like. It is not a final version just illustrative screen to give an idea.]



10-Declare winner/End game



11.2 Architecture of the graphical systems

11.2.1 Principles of the GUI

The graphical user interface is planned according to the MVC design pattern. The goal is to keep the game logic and the graphical interface separated. The existing prototype program acts as the Model, while the new GUI classes provide the View and Controller parts.

MVC Structure

Part	Responsibility
Model	The existing prototype system. It stores the game state and executes the game rules.
View	Displays the game visually through windows, panels, buttons, and drawable game elements.
Controller	Receives user actions from the GUI and calls the matching operation in the prototype system.

The GUI does not duplicate the prototype logic. For example, the GUI may display that a pipe is punctured or that a pump is broken, but the actual state is still stored in the prototype model.

View Classes

The following classes form the graphical part of the program:

Class / Interface	Purpose
ViewManager	Switches between the different screens of the program.
MainMenuView	Displays the main menu with Start Game, Instructions, and Exit options.
GameSetupView	Collects player names, team selections, and game duration.
InstructionsView	Displays the rules and basic controls.
GameView	Main gameplay screen containing the network, players, action panel, and scoreboard.
ScoreBoardView	Displays the current scores and remaining time.
ActionPanelView	Displays the available action buttons for the current player.
EndGameView	Displays the winner and final scores.
Observer	Interface for GUI elements that must update and redraw when the game state changes.
NetworkElementView	Abstract parent for visual network elements. Stores common position and drawing data.

PipeView	Draws pipes and shows states such as punctured, occupied, or free-ended.
PumpView	Draws pumps and shows working/broken state and flow direction.
SpringView	Draws springs as water source elements.
CisternView	Draws cisterns and shows available generated pipes and pumps.
PlayerView	Abstract parent for visual player tokens.
PlumberView	Draws plumber players.
SaboteurView	Draws saboteur players.

Controller Classes

The controller part connects GUI events to the prototype system.

Class / Interface	Purpose
GameController	Handles button clicks and calls the appropriate prototype operation.
MovementHandler	Interface for movement validation and movement handling.
PlumberMovementHandler	Handles valid movement for plumbers.
SaboteurMovementHandler	Handles valid movement for saboteurs.

For example, if the user clicks the Repair Pipe button, the GameController checks the selected element and calls the matching repair operation in the model. If the user selects a movement target, the correct MovementHandler checks whether the movement is possible before updating the view.

Update Principle

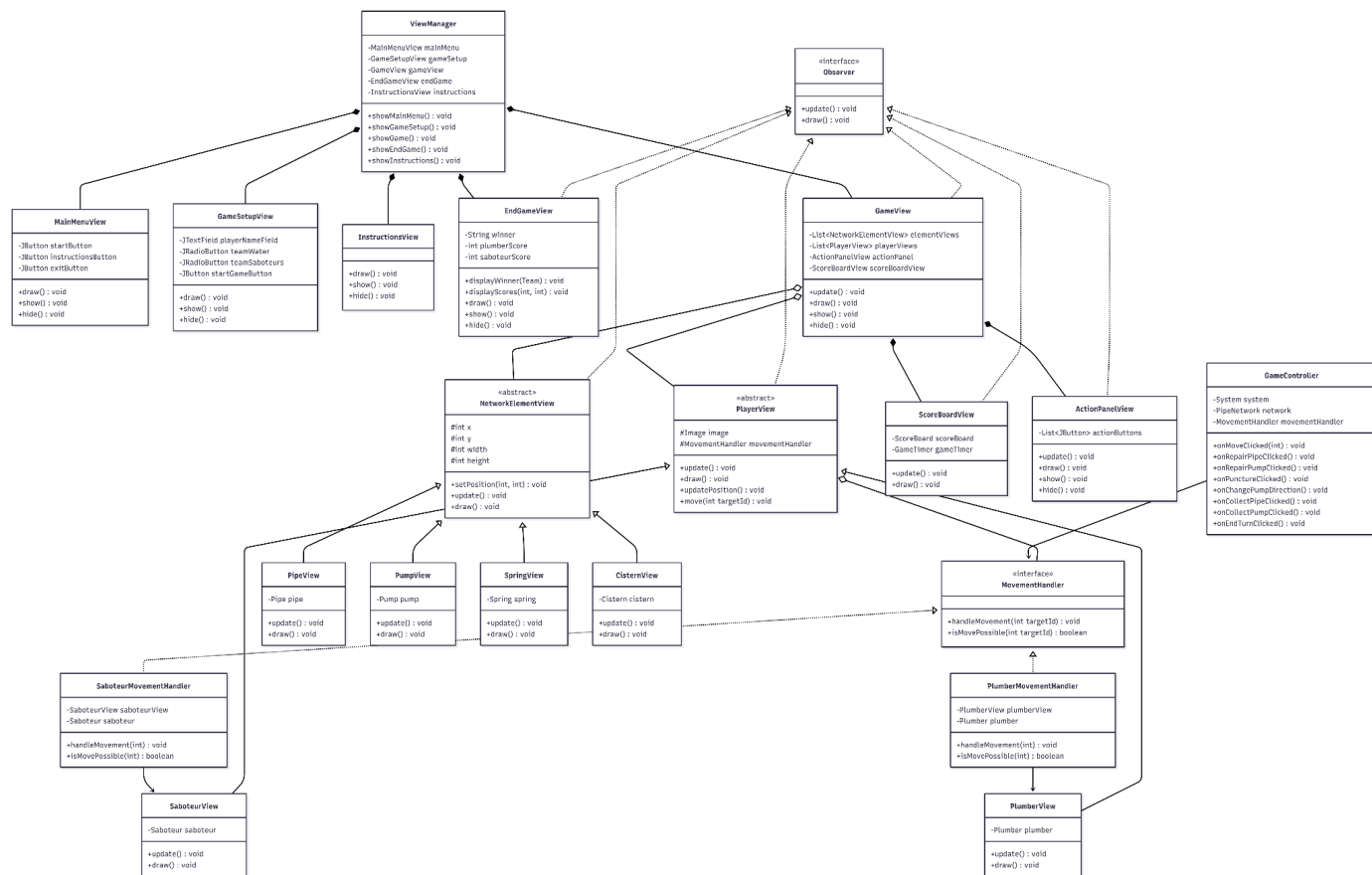
The GUI will use a mixed update model, combining push and pull methods.

In the push model, the model notifies the GUI when an important change happens. This is supported by the Observer interface. GUI classes implementing Observer can update and redraw themselves after events such as movement, repair, puncture, pump direction change, water flow recalculation, or turn change.

In the pull model, the GUI reads the current state from the model when it needs to refresh displayed information. For example, ScoreBoardView can read the current scores and timer values, while ActionPanelView can check the current player to show only the valid actions.

Using a mixed model is suitable because some changes should refresh the GUI immediately, while other information can simply be read when the screen is redrawn. This keeps the GUI synchronized with the prototype while avoiding duplicated game state inside the interface.

11.2.2 GUI Structure diagram



11.3 GUI Classes

11.3.1 ViewManager

- **Responsibility**

ViewManager manages the main GUI screens of the application. It is responsible for switching between the main menu, game setup screen, game screen, end game screen, and instructions screen.

- **Superclasses**

None specified

- **Interfaces**

None

- **Attributes**

- mainMenu: feature: stores the main menu view; type: MainMenuView; visibility: private; initial value: assigned during construction.

- gameSetup: feature: stores the game setup view; type: GameSetupView; visibility: private; initial value: assigned during construction.

- gameView: feature: stores the main game view; type: GameView; visibility: private; initial value: assigned during construction.
- endGame: feature: stores the end game view; type: EndGameView; visibility: private; initial value: assigned during construction.
- instructions: feature: stores the instructions view; type: InstructionsView; visibility: private; initial value: assigned during construction.

- **Methods**

- public void showMainMenu(): Shows the main menu screen.
- public void showGameSetup(): Shows the game setup screen.
- public void showGame(): Shows the main game screen.
- public void showEndGame(): Shows the end game screen.
- public void showInstructions(): Shows the instructions screen.

11.3.2 MainMenuView

- **Responsibility**

MainMenuView represents the main menu of the GUI. It contains the main buttons used to start the game, open instructions, or exit the application.

- **Superclasses**

None specified

- **Interfaces**

None

- **Attributes**

- startButton: feature: button for starting the game; type: JButton; visibility: private; initial value: assigned during construction.
- instructionsButton: feature: button for opening the instructions screen; type: JButton; visibility: private; initial value: assigned during construction.
- exitButton: feature: button for exiting the application; type: JButton; visibility: private; initial value: assigned during construction.

- **Methods**

- public void draw(): Draws the main menu view.
- public void show(): Shows the main menu view.
- public void hide(): Hides the main menu view.

11.3.3 GameSetupView

- **Responsibility**

GameSetupView represents the setup screen before the game starts. It allows the player to enter a name, choose a team, and start the game.

- **Superclasses**

None specified

- **Interfaces**

None

• Attributes

- playerNameField: feature: input field for the player name; type: JTextField; visibility: private; initial value: assigned during construction.
- teamWater: feature: radio button for selecting the water team; type: JRadioButton; visibility: private; initial value: assigned during construction.
- teamSaboteurs: feature: radio button for selecting the saboteur team; type: JRadioButton; visibility: private; initial value: assigned during construction.
- startGameButton: feature: button for starting the game; type: JButton; visibility: private; initial value: assigned during construction.

• Methods

- public void draw(): Draws the game setup view.
- public void show(): Shows the game setup view.
- public void hide(): Hides the game setup view.

11.3.4 InstructionsView**• Responsibility**

InstructionsView represents the instruction screen of the GUI. It is responsible for displaying the game instructions to the user.

• Superclasses

None specified

• Interfaces

None

• Attributes

None specified

• Methods

- public void draw(): Draws the instructions view.
- public void show(): Shows the instructions view.
- public void hide(): Hides the instructions view.

11.3.5 EndGameView**• Responsibility**

EndGameView represents the final screen of the game. It displays the winner and the final scores of the plumber and saboteur teams.

• Superclasses

None specified

• Interfaces

Observer

• Attributes

- winner: feature: stores the winner information; type: String; visibility: private; initial value: "".
- plumberScore: feature: stores the plumber team score; type: int; visibility: private; initial

value: 0.

- `saboteurScore`: feature: stores the saboteur team score; type: `int`; visibility: `private`; initial value: 0.

- **Methods**

- `public void displayWinner(Team team)`: Displays the winner.
- `public void displayScores(int plumberScore, int saboteurScore)`: Displays the final scores.
- `public void draw()`: Draws the end game view.
- `public void show()`: Shows the end game view.
- `public void hide()`: Hides the end game view.

11.3.6 GameView

- **Responsibility**

`GameView` represents the main game screen. It contains the network element views, player views, action panel, and scoreboard view.

- **Superclasses**

None specified

- **Interfaces**

`Observer`

- **Attributes**

- `elementViews`: feature: stores the network element views; type: `List<NetworkElementView>`; visibility: `private`; initial value: empty list.
- `playerViews`: feature: stores the player views; type: `List<PlayerView>`; visibility: `private`; initial value: empty list.
- `actionPanel`: feature: stores the action panel view; type: `ActionPanelView`; visibility: `private`; initial value: assigned during construction.
- `scoreBoardView`: feature: stores the scoreboard view; type: `ScoreBoardView`; visibility: `private`; initial value: assigned during construction.

- **Methods**

- `public void update()`: Updates the game view.
- `public void draw()`: Draws the game view.
- `public void show()`: Shows the game view.
- `public void hide()`: Hides the game view.

11.3.7 ScoreBoardView

- **Responsibility**

`ScoreBoardView` displays the current score and timer information during the game. It keeps the score and timer display separated from the main game screen.

- **Superclasses**

None specified

- **Interfaces**

`Observer`

● Attributes

- scoreBoard: feature: stores the scoreboard reference; type: ScoreBoard; visibility: private; initial value: assigned during construction.
- gameTimer: feature: stores the game timer reference; type: GameTimer; visibility: private; initial value: assigned during construction.

● Methods

- public void update(): Updates the scoreboard view.
- public void draw(): Draws the scoreboard view.

11.3.8 ActionPanelView**● Responsibility**

ActionPanelView displays the available action buttons for the player during the game. It is responsible for showing and updating the action controls.

● Superclasses

None specified

● Interfaces

Observer

● Attributes

- actionButtons: feature: stores the action buttons; type: List<JButton>; visibility: private; initial value: empty list.

● Methods

- public void update(): Updates the action panel.
- public void draw(): Draws the action panel.
- public void show(): Shows the action panel.
- public void hide(): Hides the action panel.

11.3.9 NetworkElementView**● Responsibility**

NetworkElementView is an abstract base class for network element views. It contains common position and size attributes for pipe, pump, spring, and cistern views.

● Superclasses

None specified

● Interfaces

Observer

● Attributes

- x: feature: stores the x position; type: int; visibility: protected; initial value: 0.
- y: feature: stores the y position; type: int; visibility: protected; initial value: 0.
- width: feature: stores the width; type: int; visibility: protected; initial value: 0.
- height: feature: stores the height; type: int; visibility: protected; initial value: 0.

- **Methods**

- public void setPosition(int x, int y): Sets the position of the network element view.
- public void update(): Updates the network element view.
- public void draw(): Draws the network element view.

11.3.10 PipeView

- **Responsibility**

PipeView represents the graphical view of a pipe. It uses the related Pipe object to display the pipe in the GUI.

- **Superclasses**

NetworkElementView

- **Interfaces**

Observer

- **Attributes**

- pipe: feature: stores the related pipe object; type: Pipe; visibility: private; initial value: assigned during construction.

- **Methods**

- public void update(): Updates the pipe view.
- public void draw(): Draws the pipe view.

11.3.11 PumpView

- **Responsibility**

PumpView represents the graphical view of a pump. It uses the related Pump object to display the pump in the GUI.

- **Superclasses**

NetworkElementView

- **Interfaces**

Observer

- **Attributes**

- pump: feature: stores the related pump object; type: Pump; visibility: private; initial value: assigned during construction.

- **Methods**

- public void update(): Updates the pump view.
- public void draw(): Draws the pump view.

11.3.12 SpringView

- **Responsibility**

SpringView represents the graphical view of a spring. It uses the related Spring object to display the spring in the GUI.

- **Superclasses**

NetworkElementView

- **Interfaces**

Observer

- **Attributes**

- spring: feature: stores the related spring object; type: Spring; visibility: private; initial value: assigned during construction.

- **Methods**

- public void update(): Updates the spring view.
- public void draw(): Draws the spring view.

11.3.13 CisternView

- **Responsibility**

CisternView represents the graphical view of a cistern. It uses the related Cistern object to display the cistern in the GUI.

- **Superclasses**

NetworkElementView

- **Interfaces**

Observer

- **Attributes**

- cistern: feature: stores the related cistern object; type: Cistern; visibility: private; initial value: assigned during construction.

- **Methods**

- public void update(): Updates the cistern view.
- public void draw(): Draws the cistern view.

11.3.14 PlayerView

- **Responsibility**

PlayerView is an abstract base class for player views. It contains common player display data and movement handling for plumber and saboteur views.

- **Superclasses**

None specified

- **Interfaces**

Observer

- **Attributes**

- image: feature: stores the player image; type: Image; visibility: protected; initial value: assigned during construction.
- movementHandler: feature: stores the movement handler used by the player view; type: MovementHandler; visibility: protected; initial value: assigned during construction.

- **Methods**

- public void update(): Updates the player view.
- public void draw(): Draws the player view.
- public void updatePosition(): Updates the player position.
- public void move(int targetId): Moves the player view using the target id.

11.3.15 PlumberView

- **Responsibility**

PlumberView represents the graphical view of a plumber player. It uses the related Plumber object to display the plumber in the GUI.

- **Superclasses**

PlayerView

- **Interfaces**

Observer

- **Attributes**

- plumber: feature: stores the related plumber object; type: Plumber; visibility: private; initial value: assigned during construction.

- **Methods**

- public void update(): Updates the plumber view.
- public void draw(): Draws the plumber view.

11.3.16 SaboteurView

- **Responsibility**

SaboteurView represents the graphical view of a saboteur player. It uses the related Saboteur object to display the saboteur in the GUI.

- **Superclasses**

PlayerView

- **Interfaces**

Observer

- **Attributes**

- saboteur: feature: stores the related saboteur object; type: Saboteur; visibility: private; initial value: assigned during construction.

- **Methods**

- public void update(): Updates the saboteur view.
- public void draw(): Draws the saboteur view.

11.3.17 GameController

- **Responsibility**

GameController connects user actions from the GUI to the prototype system. It handles

actions such as movement, repairing, puncturing, changing pump direction, collecting components, and ending the turn.

- **Superclasses**

None specified

- **Interfaces**

None

- **Attributes**

- system: feature: stores the system object; type: System; visibility: private; initial value: assigned during construction.
- network: feature: stores the pipe network object; type: PipeNetwork; visibility: private; initial value: assigned during construction.
- movementHandler: feature: stores the movement handler; type: MovementHandler; visibility: private; initial value: assigned during construction.

- **Methods**

- public void onMoveClicked(int targetId): Handles the move button action.
- public void onRepairPipeClicked(): Handles the repair pipe action.
- public void onRepairPumpClicked(): Handles the repair pump action.
- public void onPunctureClicked(): Handles the puncture action.
- public void onChangePumpDirection(): Handles the change pump direction action.
- public void onCollectPipeClicked(): Handles the collect pipe action.
- public void onCollectPumpClicked(): Handles the collect pump action.
- public void onEndTurnClicked(): Handles the end turn action.

11.3.18 PlumberMovementHandler

- **Responsibility**

PlumberMovementHandler handles movement for plumber players. It checks whether movement is possible and handles movement using the target id.

- **Superclasses**

None specified

- **Interfaces**

MovementHandler

- **Attributes**

- plumberView: feature: stores the plumber view; type: PlumberView; visibility: private; initial value: assigned during construction.
- plumber: feature: stores the related plumber object; type: Plumber; visibility: private; initial value: assigned during construction.

- **Methods**

- public void handleMovement(int targetId): Handles plumber movement.
- public boolean isMovePossible(int targetId): Checks whether plumber movement is possible.

11.3.19 SaboteurMovementHandler

- **Responsibility**

SaboteurMovementHandler handles movement for saboteur players. It checks whether movement is possible and handles movement using the target id.

- **Superclasses**

None specified

- **Interfaces**

MovementHandler

- **Attributes**

- saboteurView: feature: stores the saboteur view; type: SaboteurView; visibility: private; initial value: assigned during construction.

- saboteur: feature: stores the related saboteur object; type: Saboteur; visibility: private; initial value: assigned during construction.

- **Methods**

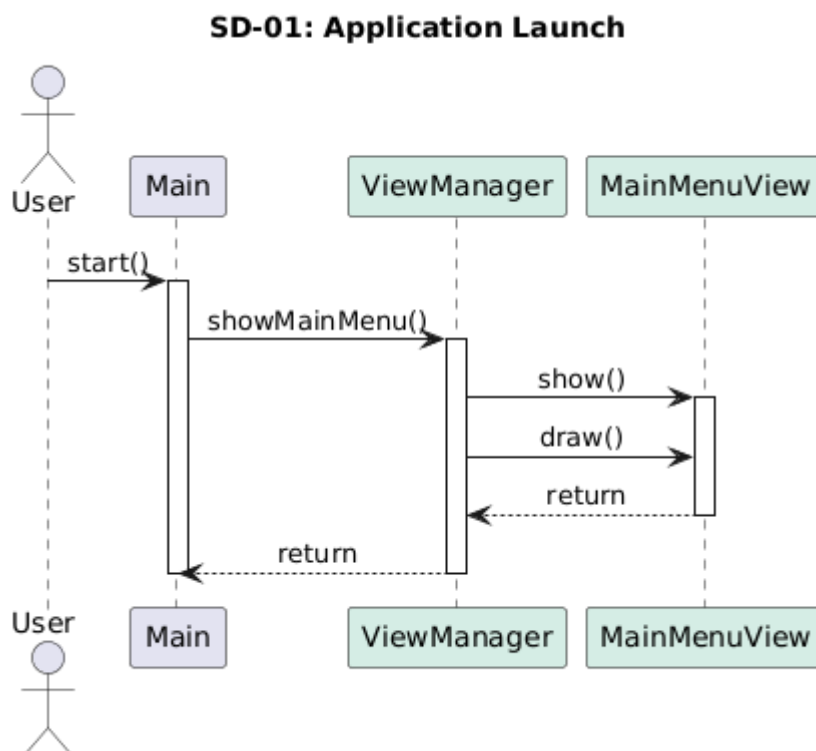
- public void handleMovement(int targetId): Handles saboteur movement.

- public boolean isMovePossible(int targetId): Checks whether saboteur movement is possible.

11.4 Dynamic connection between the model and the GUI

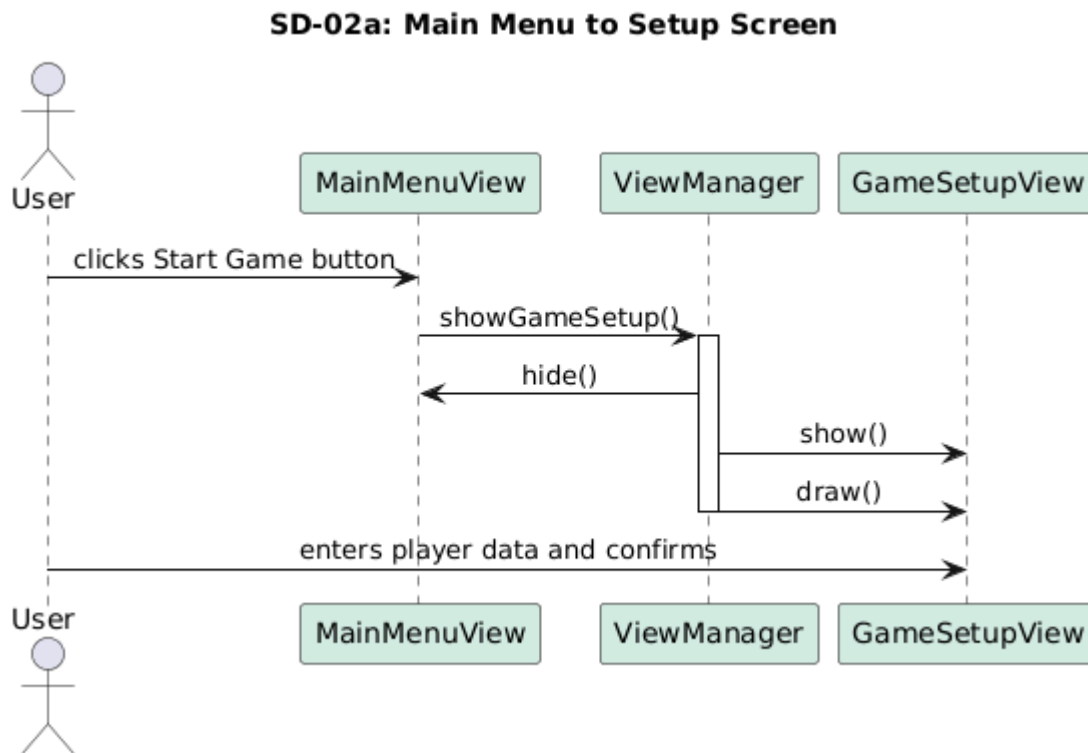
SD-01: Application Launch

Description: Describes the startup sequence of the program. When the user launches the application, Main calls showMainMenu() on ViewManager, which calls show() and draw() on MainMenuView to render the main screen.

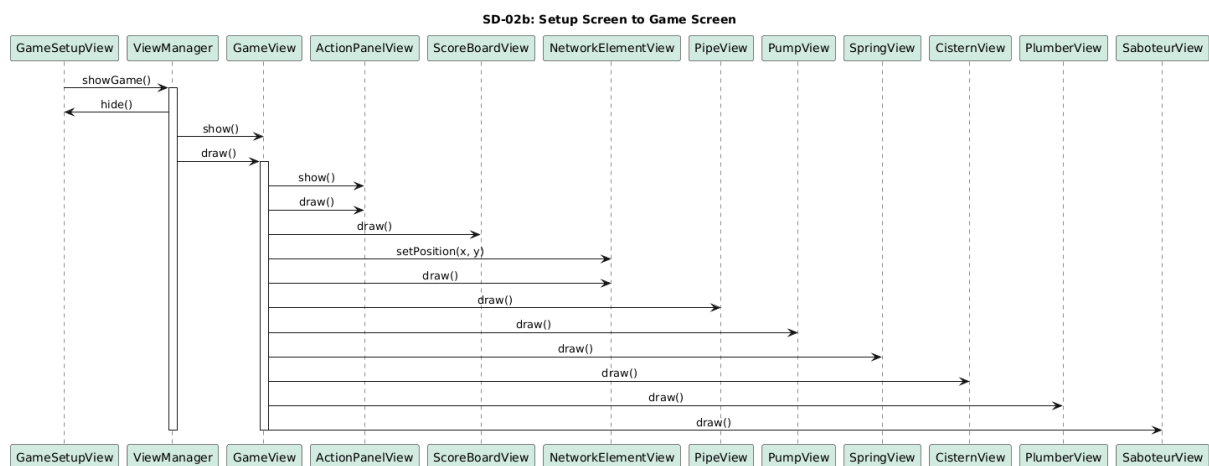


SD-02a: Main Menu to Setup Screen

Description: Describes the sequence when the user clicks Start Game on the main menu. `ViewManager` hides `MainMenuView`, then shows and draws `GameSetupView` for player data entry.

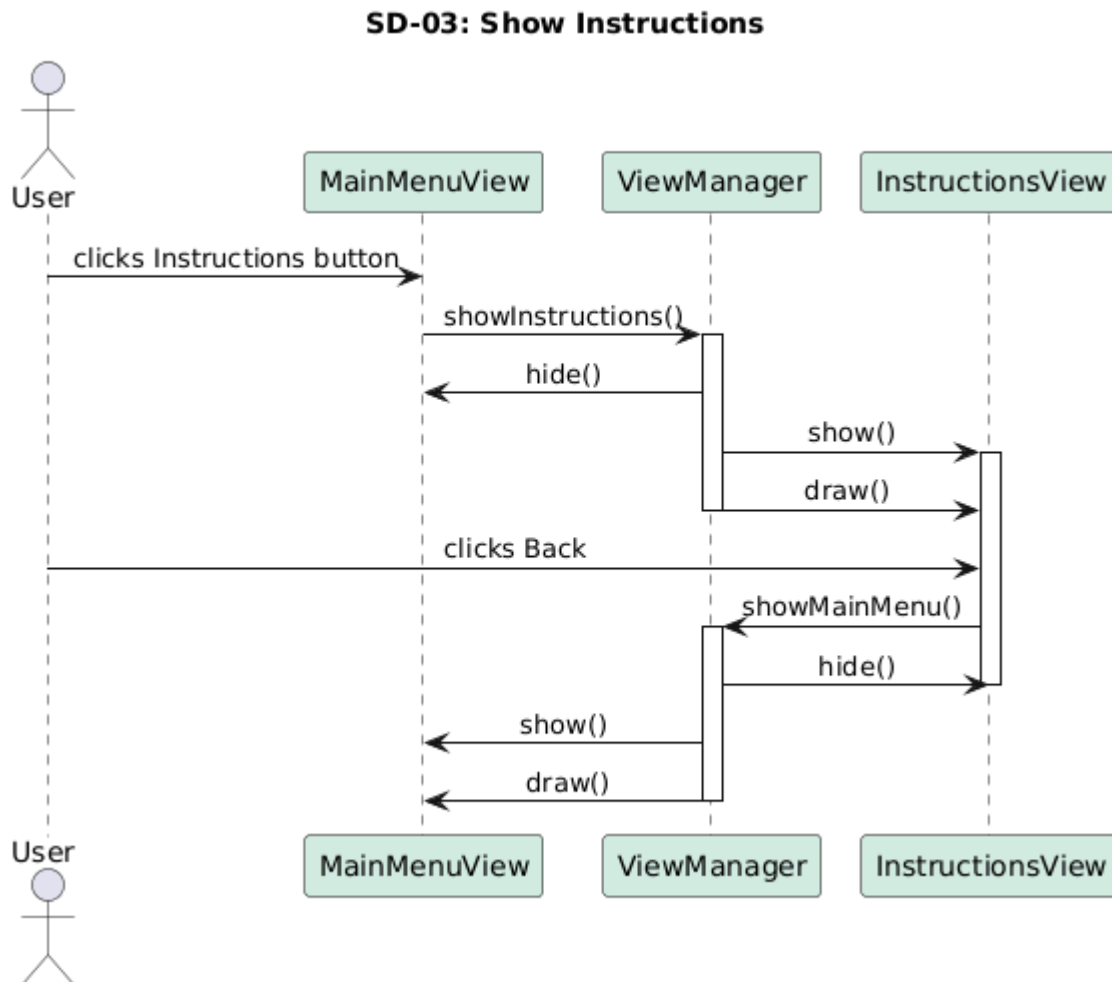
**SD-02b: Setup Screen to Game Screen**

Description: Describes the game initialization sequence after the user confirms setup. `ViewManager` hides `GameSetupView` and displays `GameView`. `GameView` then initializes and draws all sub-views: the action panel, scoreboard, each network element view, and each player view.

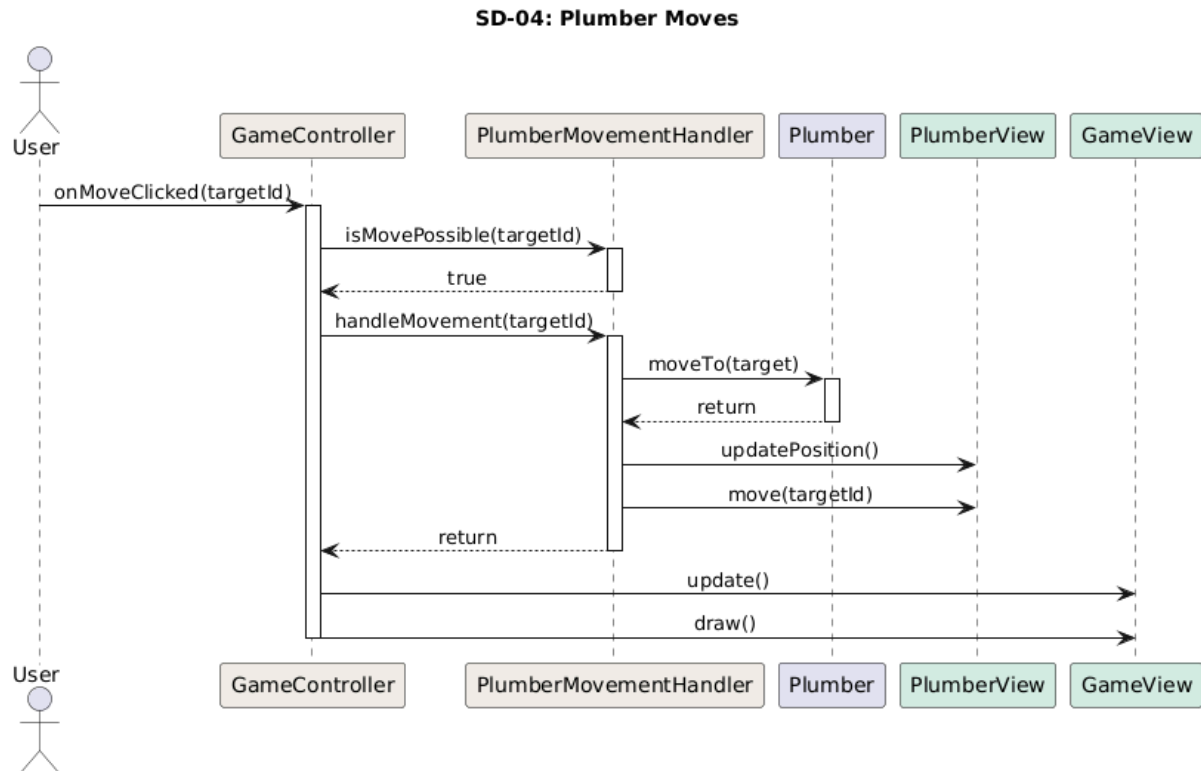


SD-03: Show Instructions

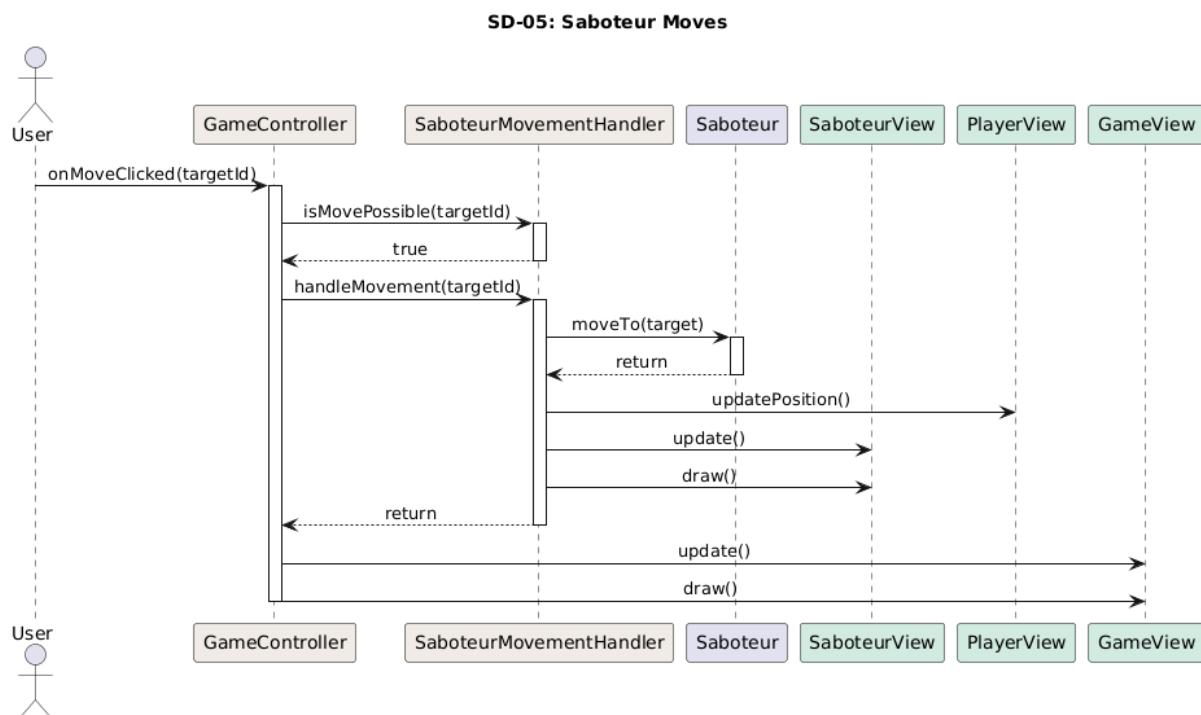
Description: Describes the sequence when the user opens and closes the instructions screen from the main menu. `ViewManager` hides `MainMenuView`, then shows and draws `InstructionsView`. When the user closes instructions, `ViewManager` hides `InstructionsView` and restores `MainMenuView`.

**SD-04: Plumber Moves**

Description: Describes the sequence when a Plumber selects Move from the action menu. `GameController` delegates to `PlumberMovementHandler` to validate and execute the move. On success, `PlumberView` updates its position and redraws. `GameView` then refreshes.

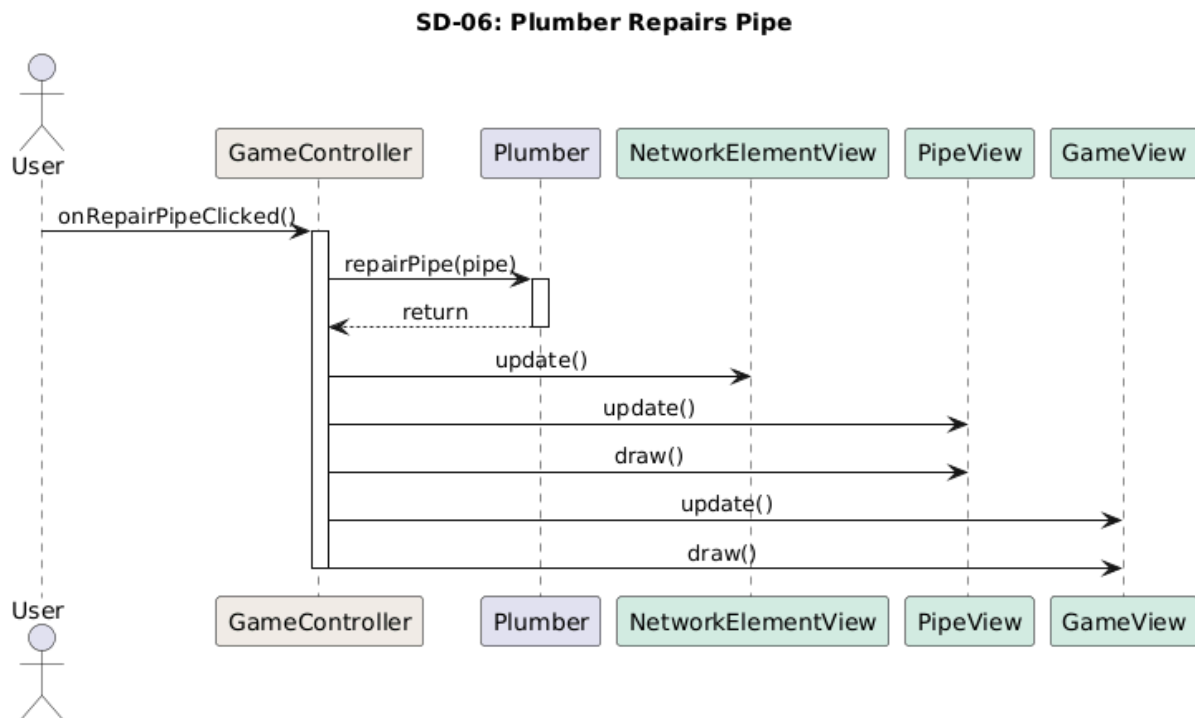
**SD-05: Saboteur Moves**

Description: Describes the sequence when a Saboteur selects Move. Mirrors the plumber movement flow but uses SaboteurMovementHandler and SaboteurView. PlayerView.updatePosition() is called to sync the visual position before redraw.

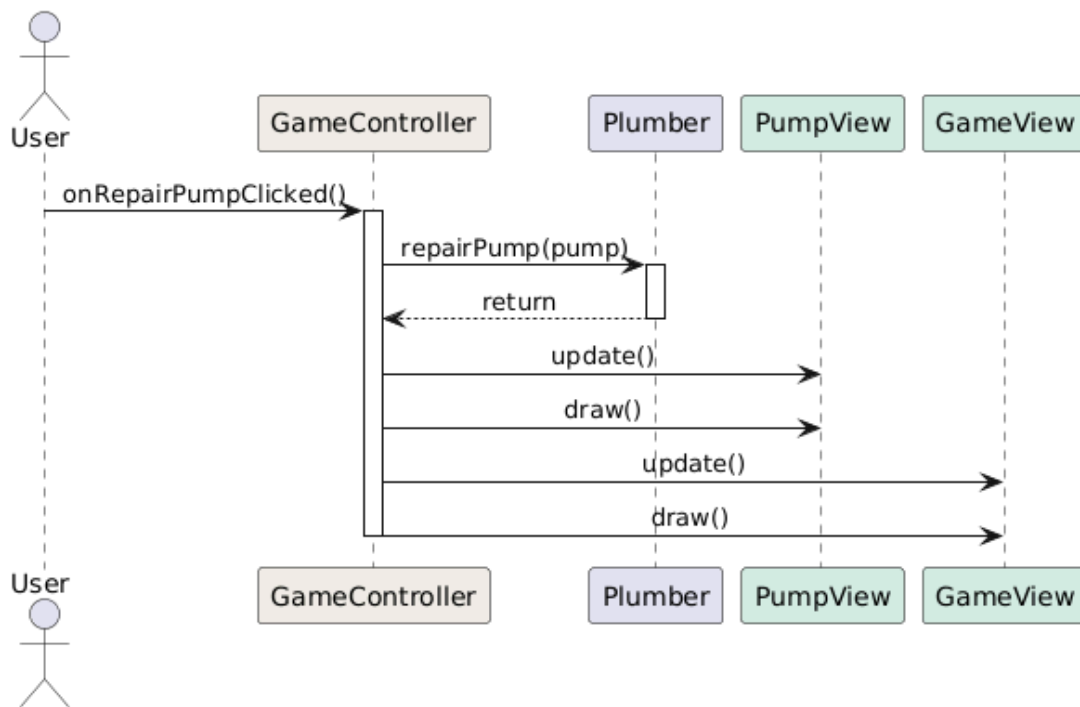


SD-06: Plumber Repairs Pipe

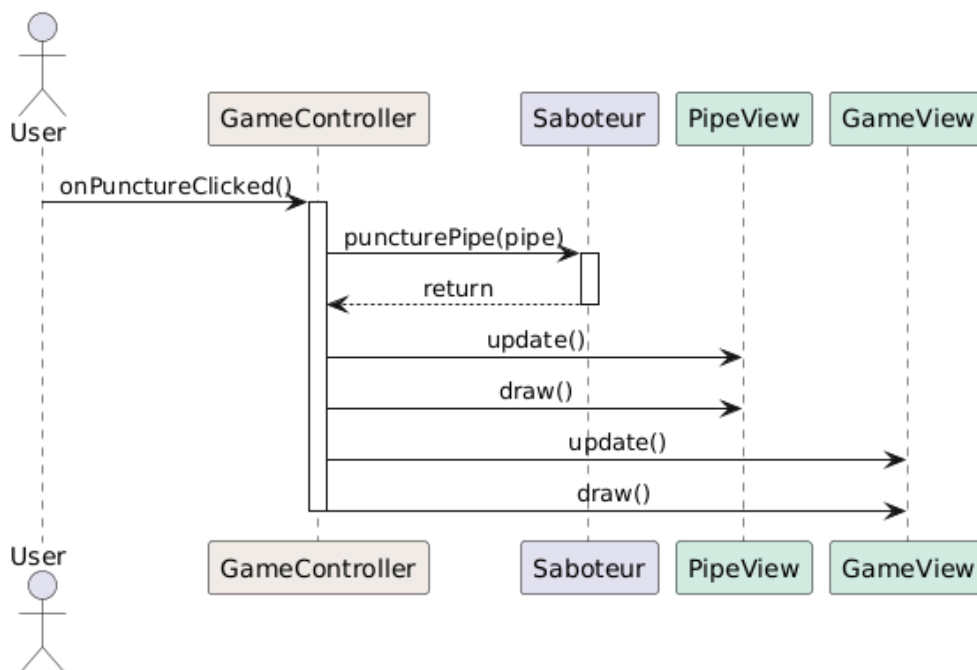
Description: Describes the sequence when a Plumber selects Repair Pipe. GameController calls `repairPipe()` on the Plumber model object. Once repaired, `NetworkElementView.update()` and `PipeView.update()` are called, followed by `PipeView.draw()` to reflect the new pipe state in the GUI.

**SD-07: Plumber Repairs Pump**

Description: Describes the sequence when a Plumber selects Repair Pump. GameController calls `repairPump()` on the Plumber model. PumpView then updates and redraws to reflect the restored pump state.

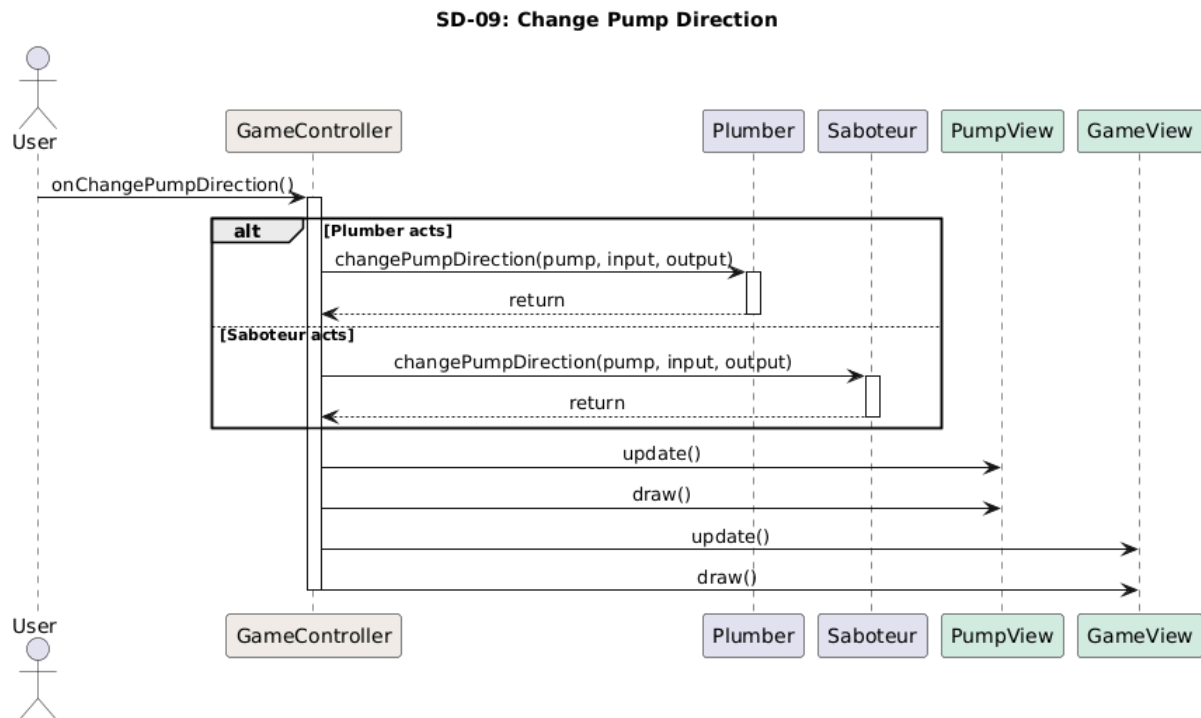
SD-07: Plumber Repairs Pump**SD-08: Saboteur Punctures Pipe**

Description: Describes the sequence when a Saboteur selects Puncture Pipe. GameController calls `puncturePipe()` on the Saboteur model. PipeView updates to reflect the punctured state and GameView redraws.

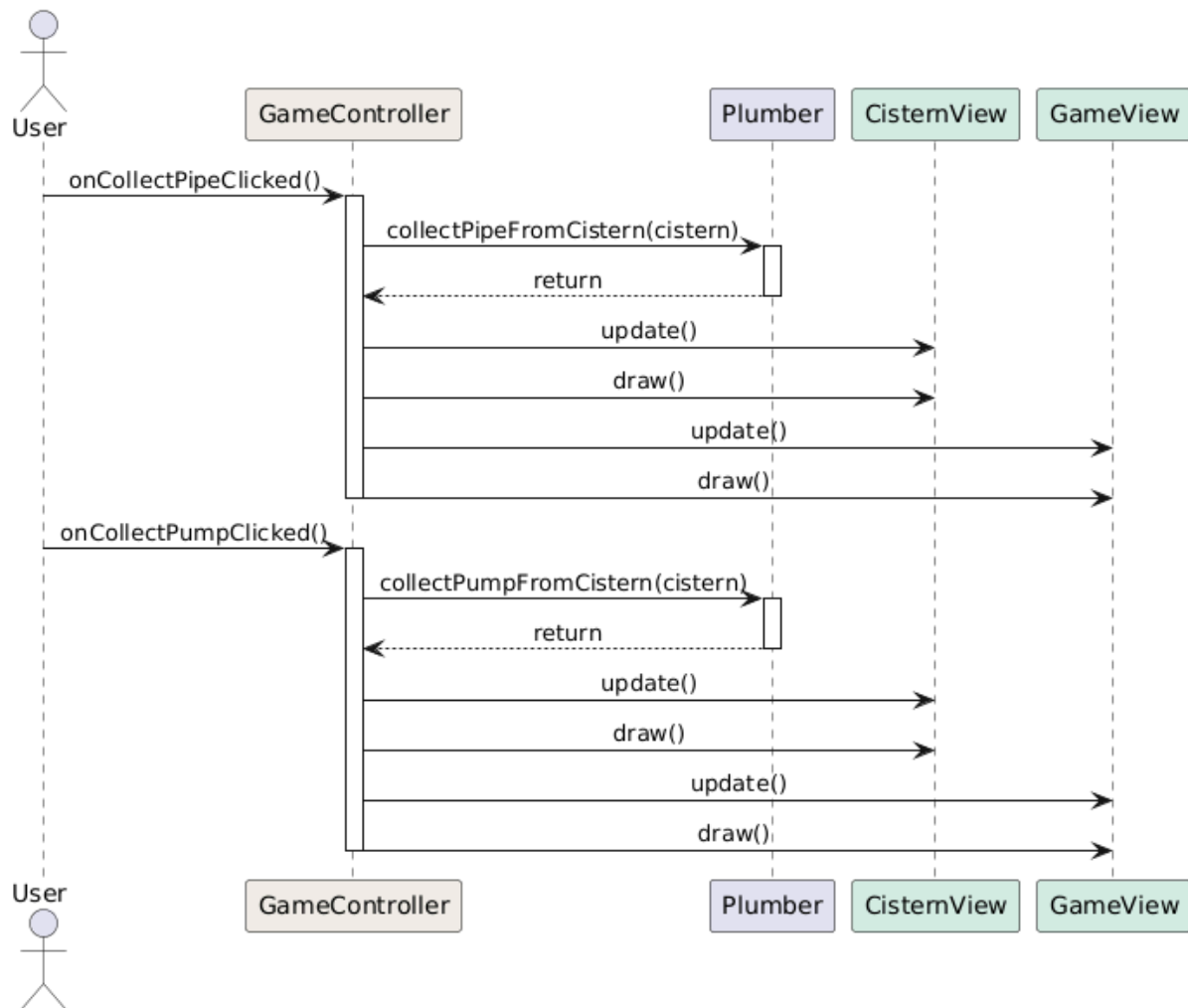
SD-08: Saboteur Punctures Pipe

SD-09: Change Pump Direction

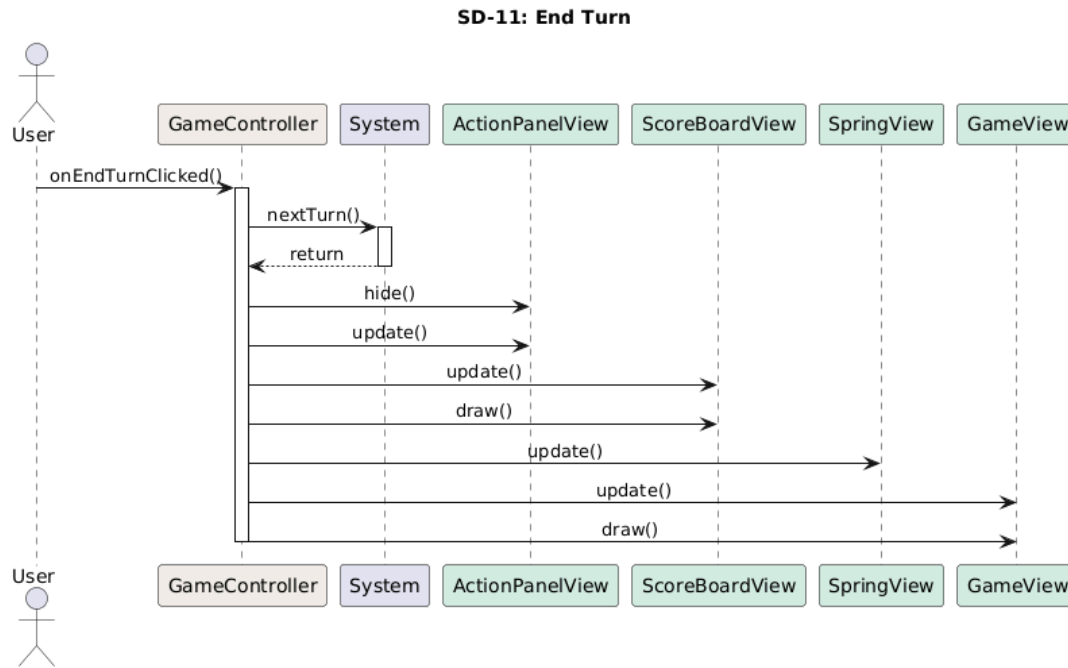
Description: Describes the sequence when a Plumber or Saboteur changes the active pump direction. GameController calls `changePumpDirection()` on the respective model object. PumpView updates to reflect the new routing and GameView redraws.

**SD-10: Collect Component from Cistern**

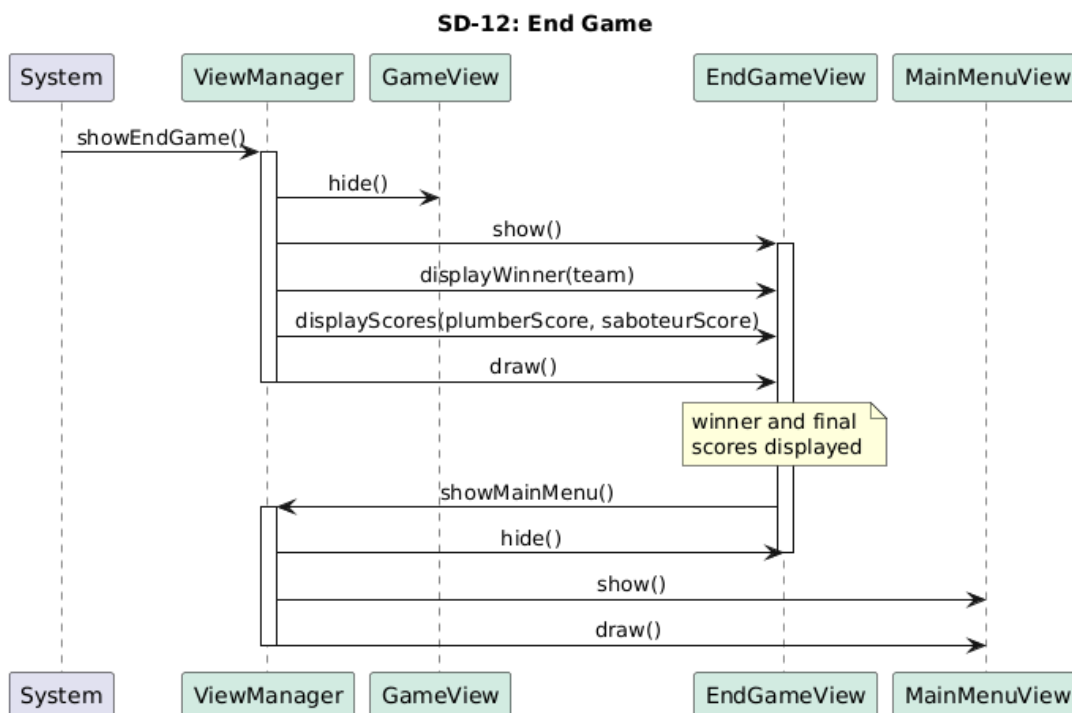
Description: Describes the sequence when a Plumber collects a pipe or pump from a cistern. GameController calls the appropriate collect method on Plumber. CisternView updates to reflect the reduced inventory count and GameView redraws.

SD-10: Collect Component from Cistern**SD-11: End Turn**

Description: Describes the sequence when the current player ends their turn. GameController calls `nextTurn()` on System, which advances to the next player. ActionPanelView hides and updates for the new player, ScoreBoardView updates with the latest scores, and SpringView updates after water flow recalculation.

**SD-12: End Game**

Description: Describes the sequence when the game timer expires and the game ends. `System.endGame()` is called, which triggers `ViewManager` to switch to `EndGameView`. The winner and final scores are displayed. `GameView` is hidden and `EndGameView` shows and draws the result screen.



11.5 Tools Used

Tool	Purpose	Description
ChatGPT (AI)	Writing assistance	Used for proofreading, grammar correction, brainstorming, and clarifying design decisions across sections 11.2, 11.3, and 11.4.
PlantUML	Sequence diagram creation	Used to generate all sequence diagrams in section 11.4 from textual UML notation.
Draw.io	Class diagram creation	Used to create the GUI structure diagram in section 11.2.2.
Microsoft Word	Documentation	Used to write, format, and organize the GUI specification document.
Discord	Communication	Used for online team meetings and coordination.
Google Meet	Communication	Used for real-time collaboration during working sessions.
GitHub	Version control	Used to manage and share source code and documentation.
Microsoft Teams	Communication	Used for file sharing and informal coordination.

11.6 Protocol

Start (date & time)	Duration (hours)	Performer(s) name	Activity description
07.05.2026 18:00	1 hour	All members	Meeting about the structure of the GUI specification document. The team reviewed the requirements of Template 11 and discussed how the graphical interface should reflect the existing prototype system. Decisions were made about responsibility distribution: İlgin prepares the graphical user interface section (11.1), Hameez prepares the GUI architecture principles (11.2.1), Yahya prepares the GUI structure diagram (11.2.2), Aasif prepares GUI class specifications (11.3), and Ibrahim prepares the dynamic GUI-model interaction diagrams (11.4). Arda is focusing on standardizing the code

			structure of the Prototype program to implement it later. The team also agreed to keep the GUI design consistent with the previously implemented prototype classes and use cases.
07.05.2026 20:30	4 hours	Ilgin Tunç	Preparation of section 11.1 – Graphical User Interface. GUI screen layouts and menu structures were designed based on the existing prototype functionality. Draft screenshot-style interface sketches were created for the main menu, game screen, player actions, and status displays. The visual structure was designed to remain consistent with the current console-based prototype logic and gameplay flow.
08.05.2026 14:00	2 hours	Muhammad Hameez Khan	Preparation of section 11.2.1 – Principles of the GUI. The MVC-based connection between the GUI and the prototype system was planned. The responsibilities of the model, view, and controller layers were described. Decisions were documented regarding how GUI elements will interact with the game logic classes and how updates from the model will be reflected in the interface.
08.05.2026 16:30	2 hours	Yahya Akhrikhar	Preparation of section 11.2.2 – GUI Structure Diagram. A class diagram containing the planned GUI classes and their connections to the prototype classes was created. The structure diagram was designed to remain consistent with the existing class hierarchy and gameplay architecture developed during the prototype phase.
09.05.2026 13:00	3 hours	Aasif Mohd	Preparation of section 11.3 – GUI Classes. Responsibilities, attributes, methods, and relationships of the planned GUI classes were specified. The modified prototype classes affected by the GUI integration were identified and documented. Method responsibilities and interactions between interface components and game logic were described.
09.05.2026 17:30	5 hours	Muhammad Ibrahim Shueb	Preparation of section 11.4 – Dynamic Connection Between the Model and the

			GUI. Sequence diagrams describing interactions between GUI components and the game model were created. Dialogs for menu handling, player actions, game updates, and visual refresh operations were specified consistently with the GUI class structure and existing prototype methods.
10.05.2026 19:00	2 hours	All members	Final review meeting of the GUI specification document. The team checked consistency between sections 11.1–11.4, verified that the GUI plans remained compatible with the implemented prototype program, and corrected formatting and terminology inconsistencies. Final decisions regarding naming conventions, GUI behavior, and document organization were approved before submission.